Software Developer – Technical Exercise

Take-home exercise: Grant Task Tracker (full stack, AI-assisted)

Time to spend	We suggest about **2 hours**. We don't expect everything to be finished, so prioritise, and tell us what you left out and why. There's no time limit, so you're welcome to spend longer polishing if you want to.
Deadline	Within **7 calendar days** of receiving this document. Work whenever suits you.
AI tools	**Encouraged.** Use Claude, Claude Code, ChatGPT, GitHub Copilot, Cursor or similar.
Tech stack	C# / .NET (ASP.NET Core Web API, any current version), Entity Framework Core, React + TypeScript
Follow-up	A 1-hour session where you present your solution (details in section 8)

What we're looking for. We use AI coding tools every day, so this exercise is designed to be done with one. We're assessing your judgement more than the amount of code: how you break the problem down, how you guide the AI, how you check its output, and whether you can explain and defend every line you submit. **A small, correct, well-tested solution beats a large, untested one.** If you leave anything unfinished, write down what you would do next.

1. Background

Imagine a community grants program that funds local not-for-profit organisations. When an organisation applies, a **grant** record is opened and a grants officer manages it until it is closed. Along the way the officer creates **tasks**, such as "Check eligibility documents", "Verify bank details" or "Review progress report".

Grants officers need a simple tool to see their grants, add tasks, mark tasks as complete and spot overdue tasks. Your job is to build a small version of this tool.

Each grant has a reference number, the applying organisation's name, an application date, and is either open or closed. Each task has a title, a due date and a priority (Low, Medium or High), and can be marked as completed.

The scenario, organisations and all data in this exercise are fictional.

2. Back end requirements (ASP.NET Core Web API)

2.1 Endpoints

Endpoint	Purpose
`GET /grants?status=Open`	List grants, with an optional status filter. Include the number of pending tasks for each grant.
`GET /grants/{id}`	Get one grant with its tasks.
`POST /grants/{id}/tasks`	Add a task to a grant.
`PUT /tasks/{id}/complete`	Mark a task as completed.
`PUT /grants/{id}/close`	Close a grant.
`GET /tasks/overdue`	List all overdue tasks across all grants, with the most overdue first.

You can choose the route names, HTTP methods and response shapes, but be ready to explain your choices.

2.2 Business rules

Rule	Description
BR-1	A task **cannot be added to a closed grant**.
BR-2	A new task's **due date** cannot be in the past, and cannot be before the grant's application date.
BR-3	The task title is required and must be 100 characters or fewer.
BR-4	Completing a task records the date it was completed. Completing a task that is already completed must **not** change that date. Decide what the API returns in that case and explain why.
BR-5	A grant **cannot be closed while it has any pending High-priority task**.
BR-6	A task is **overdue** when it hasn't been completed and its due date is before today. "Today" must be testable, so don't call `DateTime.Now` directly inside your business logic.

2.3 Technical expectations

- **Design the data model yourself**, and be ready to explain it.
- Use EF Core with **SQLite** or the EF Core in-memory provider, so the solution runs with no external setup. Explain in your README anything you would change before running it for real.
- Return validation and business-rule errors with appropriate HTTP status codes and a consistent error body (for example, `ProblemDetails`).
- Separate the API, business logic and data access in a way you can justify. Keep it simple; don't over-engineer.
- **Automated tests**: see section 4.
- Seed the database at start-up with the data in section 6.

3. Needs from the operations team

The tool will be used by grants officers in several regional offices. The operations team, who will run and support it, have shared the needs below.

These needs are deliberately brief, and they aren't a complete list. Part of this exercise is thinking ahead: what is likely to go wrong once real people use the tool every day, and what would the team need when it does? **Build at least two of the needs below**, choosing the ones you think matter most. In your README, explain why you chose them and how you would handle the rest, along with anything else you think is missing from the list.

Need	Description
OP-1	Usage peaks sharply during funding rounds, so the team will run **several copies of the API side by side** and may restart any of them at any time without warning. Officers must not lose work or notice when this happens.
OP-2	Up to **100 officers use the tool at the same time** across all offices, and each grant is usually handled by a small team of officers rather than one person. The business rules in section 2.2 must hold at all times.
OP-3	Internet connections in regional offices are unreliable, so the same request may be sent more than once. Repeating a request must be safe, and the screen must tell officers clearly when the server can't be reached, rather than failing silently.
OP-4	Once the tool is live, a small support team who didn't build it will look into problems that officers report, often days after they happened.
OP-5	The operations team needs an automated way to tell whether each running copy of the API is working and can reach its data, so they can stop sending officers to a copy that isn't.
OP-6	The program manager wants to know how many tasks are created and completed each day, and whether the overdue list gets slower as data grows, without asking a developer to run database queries.

4. Testing

We want to understand **how you decide what to test**, not just see that tests exist. Choose your own approach and tools (for example NUnit or xUnit, Vitest or Jest, Playwright) and be ready to explain your choices.

4.1 Minimum requirements

- Automated tests for the business rules BR-1 to BR-6, including the date logic.
- Tests at **more than one level**, for example business logic on its own, the API with its database, or the front end. You decide which levels and how much to test at each.
- At least one test for a risk you identified from section 3. If a risk is impractical to test within the exercise, explain how you would test it.
- All tests run with **one command** and pass every time. They must not depend on the actual date, or on the order the tests run in.

4.2 Testing strategy

Write a short testing strategy, about one page, as a section of your README or in a separate `TESTING.md`. It should cover:

- what you test at each level, and why you split it that way
- what you deliberately didn't test, and the risk that leaves
- how you checked that tests written with AI help would actually catch a bug (for example, breaking the code on purpose and watching a test fail)
- what further testing you would add before the tool goes live

5. Front end requirements (React + TypeScript)

Keep the front end small: **one screen** for working on a grant's tasks is enough.

- A simple way to choose a grant, such as a drop-down list.
- The grant's tasks, with **overdue tasks clearly highlighted**.
- An **Add task** form with client-side validation. It must also show errors returned by the API (for example, BR-1 and BR-2).
- A **Complete** button on each pending task.
- Use any tooling (Vite is fine) and any component library, or none. Function matters more than appearance.

6. Seed data

Set due dates relative to the date the app starts, so that the overdue logic can always be demonstrated.

Grant reference	Organisation	Application date	Status	Tasks (title / priority / due / status)
GR-100001	Riverside Community Garden Inc.	today − 60 days	Open	Verify bank details / High / today − 3 days / Pending Request quote for tools / Medium / today + 5 days / Pending Check eligibility documents / High / today − 50 days / Completed
GR-100002	Northside Youth Sports Club	today − 20 days	Open	Review progress report / Low / today − 1 day / Pending Schedule site visit / Medium / today + 10 days / Pending
GR-100003	Harbour Arts Collective	today − 200 days	Closed	Final acquittal review / High / today − 150 days / Completed
GR-100004	Hillview Library Friends Group	today − 5 days	Open	*(no tasks)*

Optional extras (if you'd like to go further; they won't make up for gaps in the core requirements)

- A grants list screen with an Open / Closed / All filter, and a **Close grant** button that shows the reason if the API refuses (BR-5)
- More of the operations team's needs from section 3
- Swagger / OpenAPI, pagination
- A Dockerfile or `docker-compose` file, or a GitHub Actions workflow that builds and runs the tests

7. What to submit

1. **Source code**: a GitHub repository link (public, or private with access granted to the account the recruiter gives you) or a zip file. **Please keep your full commit history.** Don't squash it, because we want to see how the work progressed.
2. **README.md** covering:
   - how to run the API, the front end and the tests (we'll follow it exactly)

    - your testing strategy (see section 4.2), unless you put it in `TESTING.md`
    - design decisions and trade-offs
    - assumptions you made where the requirements were unclear
    - the problems you expect once the tool is in daily use, and what you did (or would do) about each one
    - what you would do next if you continued
    - roughly how long you spent (this is for context only and isn't scored)
3. **Your AI agent setup, committed to the repository**: the files you used to set up and guide your AI tool, for example `CLAUDE.md`, `AGENTS.md`, `.github/copilot-instructions.md`, Cursor rules, custom commands, skills, sub-agent definitions, specs or plans. Remove any secrets, tokens or personal details first. If you used a chat-only tool such as ChatGPT, include the instructions or context you gave it instead.
4. **AI_USAGE.md**: a short, honest account of how you used AI:
    - which tools and models you used, and for which parts of the work
    - how you set up your agent and why: what context and rules you gave it, any tool or permission settings, and how you kept it focused as the work grew
    - your workflow: how you broke the work into steps (for example plan, build, review, test), and where you used sub-agents or separate sessions, if any
    - a few key prompts, copied into the file (you don't need to send full chat transcripts or share links)
    - **at least one example where the AI was wrong, incomplete or over-engineered**, how you noticed and what you changed
    - which parts you wrote or substantially rewrote yourself

> **Ground rules.** The work must be your own, but AI help is welcome. You must be able to explain every line you submit. Don't use or paste any employer's or client's confidential code or data in your prompts. This exercise is fictional, so you don't need any. If something is unclear, make a reasonable assumption and write it down in your README.

8. Presentation session (1 hour)

Time	Activity
15 min	**Demo and walkthrough.** Run the app, then show the structure of your solution, your tests and how you handled the operations team's needs. No slides needed.
10 min	**How you worked with AI.** Show us your agent setup and walk us through your `AI_USAGE.md`: your approach, what worked, and what went wrong.
20 min	**Live change.** We'll ask for a small change or new business rule. You can use your AI tool while sharing your screen, and we'd like you to talk us through your thinking as you go.
15 min	**Q&A**: our questions about your choices, then your questions for us.

Please have the solution running on your machine before the session, with your AI tool of choice ready to use.

9. How we assess it

Area	Weight	What good looks like
Correctness and business rules	15%	All rules BR-1 to BR-6 are handled correctly, edge cases are considered, and the README instructions work.
Code quality and design	15%	Clear structure and naming, with an appropriate level of abstraction. Error handling is consistent and the API design is sensible.
Testing strategy and tests	20%	A clear, proportionate strategy with the right tests at the right levels. Tests are meaningful, readable and reliable, cover the business rules and the risks identified, and the strategy is honest about what isn't tested.
Operations team's needs	10%	Chooses sensible needs to build and builds them well. Anticipates realistic problems, including ones not listed in OP-1 to OP-6, and explains the rest in proportion.
Front end	10%	The task screen works end to end, shows API errors to the user, and uses clean, typed React code.
Effective and responsible AI use	20%	A thoughtful agent setup with clear context and rules. Breaks the work down sensibly, reviews and verifies the AI's output, and is honest about its limits.
Communication and live change	10%	Explains trade-offs clearly, makes the live change calmly and correctly, and handles questions well.

Please send any questions about this exercise to your recruitment contact. Good luck, and we look forward to seeing your work.